

1. Architecture & Foundation Setup

- **Framework & Tooling:** Next.js (App Router), TypeScript, Tailwind CSS, Lucide React icons, and shadcn/ui base primitives.
- **State & Query Management:** TanStack Query (React Query) for server state caching, pagination, sorting, and optimistic updates.
- **URL State Synchronization:** Synchronize table filters, debounced search, and pagination state with URL Search Params using backend snake_case parameters (e.g., `?page=1&limit=25&department=Engineering&sort_by=base_salary&sort_order=desc`).
- **Validation & Forms:** react-hook-form integrated with zod for form schemas strictly matching backend constraints (including positive numeric values, ISO currency codes, and status enums).
- **API Client Layer:** Typed Axios/Fetch client with native snake_case contract alignment with FastAPI OpenAPI schemas.

2. Component Breakdown & View Hierarchy

```
frontend/
├── app/
│   ├── layout.tsx      # Global layout & QueryClientProvider
│   └── page.tsx        # Main HR Dashboard (KPIs + Analytics + Data Grid)
├── components/
│   ├── dashboard/
│   │   ├── kpi-metrics-strip.tsx # Top-level summary cards (USD normalized totals, avg,
│   │   │                       median)
│   │   └── compensation-charts.tsx # Departmental & Regional distribution charts (USD
│   │   │                       aggregated)
│   └── employees/
│       ├── employee-data-table.tsx # TanStack Table wrapper with server-side pagination
│       │   & sorting
│       ├── table-toolbar.tsx      # Debounced search, faceted filters, and CSV streaming
│       │   trigger
│       ├── employee-modal.tsx     # Add/Edit employee modal (with full field coverage
│       │   including status)
│       ├── delete-dialog.tsx      # Soft-delete confirmation alert dialog
│       └── ui/                    # shadcn/ui components (Card, Dialog, Table, Select, Skeleton,
│       │                       etc.)
├── hooks/
│   ├── use-employees.ts          # TanStack Query hooks for employee CRUD & pagination
│   └── use-analytics.ts          # TanStack Query hooks for summary, department, &
│       country metrics
├── lib/
│   ├── api-client.ts            # Base API client configured with FastAPI base URL
│   ├── validations/
│   │   └── employee.ts          # Zod validation schema matching backend Pydantic models
└── __tests__/
```

└─ employee-table.test.tsx	# Unit/Integration tests for table rendering & pagination
└─ employee-form.test.tsx	# Form validation & submission tests with MSW

3. Implementation Phases

Step 1: Dashboard Analytics & KPI Visualizations ("How ACME Pays")

- **Summary KPI Cards:** Render cards using shadcn/ui Card primitives consuming GET `/api/analytics/summary`:
 - Total Annual Payroll (salary_usd aggregated).
 - Average Base Salary & Median Salary (USD).
 - Active Headcount & Currency Distribution breakdown.
- **Analytical Visualizations:**
 - **Departmental Breakdown:** Horizontal Bar Chart (Recharts) comparing total payroll budget and average compensation across departments using `/api/analytics/by-department`.
 - **Geographic Distribution:** Grouped Bar or Donut Chart showing compensation distributions across operating countries using `/api/analytics/by-country`.

Step 2: Server-Side Data Grid (TanStack Table & shadcn/ui)

- **Server-Driven Pagination & Sorting:** Configure TanStack Table in manual mode (manualPagination: true, manualSorting: true, manualFiltering: true).
- **Parameter Contract Alignment:** Bind table column sort triggers directly to snake_case backend parameters (base_salary, bonus, salary_usd).
- **Faceted Filtering Controls:** Multi-select dropdowns for country, department, employment_type, and status.
- **Debounced Search:** Text search by employee name with a 300ms debounce piped to the backend search query parameter.
- **Dual Currency Display:** Render native salary amounts alongside converted salary_usd values in employee row items.
- **CSV Export Action:** Direct toolbar button triggering backend streaming download via GET `/api/employees/export` with active filter parameters.

Step 3: CRUD Workflows & Form Validation

- **Add / Edit Modal (shadcn/ui Dialog or Sheet):**
 - **Complete Field Coverage:** name, email, country, department, job_title, employment_type, status (Active, On Leave, Terminated), currency (USD, EUR, GBP, etc.), base_salary, and bonus.
 - **Zod Schema Rules:** Enforce required strings, valid email formats, positive numeric bounds for salaries/bonuses, standard currency code lists, and strict enum checking matching backend Pydantic schemas.
- **Soft Delete Flow:** Destructive action menu triggering an alert dialog to confirm soft deletion (DELETE `/api/employees/:id`), updating cache optimistically.

Step 4: Loading, Error, Edge States & Toast Feedback

- **Skeleton Loaders:** Skeleton layouts for metric cards, chart panels, and table rows during TanStack Query fetching.
- **Edge States:** Zero-result empty states for non-matching filter criteria and error

boundaries for API failures.

- **Toast Notifications:** Sonner toast feedback on create, update, delete, and CSV export failures/successes.

Step 5: Frontend Testing & Deterministic Verification

- **Unit Testing (Vitest + React Testing Library):** Test Zod schema validation rules against edge-case numeric inputs (negative numbers, empty strings, invalid currency codes).
- **Integration Testing (Mock Service Worker - MSW):**
 - Mock FastAPI endpoints (`/api/employees`, `/api/analytics/*`) to test debounced table searching, page switching, and filter state changes in isolation.
 - Test optimistic UI mutations and error rollbacks on failed API responses.